

Practice 4

Deadline: 2 weeks from now. Should be checked onsite (during labs).

In this task, you'll be using Java 8 Stream to perform simple data analysis. Specifically, we provide a `cities.txt` and an incomplete `CityAnalysis.java` which reads `cities.txt` into a stream of cities (`Stream<City>`). You need to complete the 5 **TODOs** in `CityAnalysis.java`:

```
public static Stream<City> readCities(String filename) throws IOException
{
    return Files.lines(Paths.get(filename))
        .map(l -> l.split(", "))
        .map(a -> new City(a[0], a[1], Integer.parseInt(a[2])));
}
```

1. Count how many cities there are for each state.

The result is `Map<String, Long>`, where the key is the name of the state while the value is the number of cities in that state.

Hint: `Collectors.groupingBy, Collectors.counting()`.

2. Count the total population for each state.

The result is `Map<String, Integer>`, where the key is the name of the state while the value is the population of that state (i.e., sum of the population of each city in that state).

Hint: `Collectors.groupingBy, Collectors.summingInt()`.

3. For each state, get the city of the longest name.

The result is `Map<String, String>` or `Map<String, Optional<String>>`. Key: state, Value: city name with the longest length.

Hint:

- Start with `Collectors.groupingBy`, using the state as the key.
- `groupingBy` needs a downstream collector to decide what to do with the cities in each state. Use `Collectors.mapping` to map each `City` into just its name.
- The mapping itself also needs a downstream collector: here you can use `Collectors.maxBy`, which requires a `Comparator`. In this case, compare the names by their length `Comparator.comparingInt(String::length)`.
- So the nesting looks like: `groupingBy(state, mapping(name, maxBy(comparator)))`

4. For each state, get the set of cities with >500,000 population.

The result is `Map<String, Set<City>>`, where key is the state and value is the set of cities that satisfy the criterion. Return only the top 3 entries with the largest size of city set.

Hint:

```
Map<String, Set<City>> top3ByLargeCities = cities
    .filter(_____) // filter cities with population > 500k
    .collect(Collectors.groupingBy(
        _____, // group by state
        _____ // collect cities into a Set per state
    ))
    .entrySet()
    .stream()
    .sorted(_____) // sort by set size in descending order
    .limit(3) // top 3 states
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (a, b) -> a, // keep the first value if duplicate key is found
        LinkedHashMap::new // preserve order
    ));
```

5. Calculate the average, min, and max population for each state

Return `Map<String, List<Double>>` with key as state and values as `[avg, min, max]`. Return only the top 5 results of map sorted by state in alphabetical order.

Hint:

- Use `Collectors.groupingBy` with the state as the key.
- For the downstream collector, you'll need to combine average, min, and max of the population.
- `Collectors.summarizingInt()` can give you count, sum, min, max, and average in one shot.
- From the summary statistics, extract the values you need and put them into a List. Consider `Collectors.collectingAndThen`, which post-processes the statistics into a `List<Double>` containing `[avg, min, max]`.
- So the structure looks like: `groupingBy(state, collectingAndThen(summarizingInt(population), stats -> List.of(avg, min, max)))`.

Note

- Check out `Collectors.groupingBy` and other available collectors in `java.util.stream.Collectors`.
- You may want to override the `toString` method for `City` to facilitate printing `City` objects.
- Your output doesn't have to have the exact same format as our sample output.

Sample Output

Q1: # of cities per state:

```
{DE=1, HI=1, TX=63, MA=22, MD=5, ME=1, IA=10, ID=5, MI=24, UT=12, MN=17, MO=16, IL=29, IN=17, MS=3, A
```

Q2: population per state:

```
{DE=71292, HI=345610, TX=13748465, MA=2403297, MD=869891, ME=66214, IA=897519, ID=489295, MI=2675132,
```

Q3: longest city name per state:

```
DE:Wilmington, HI:Honolulu, TX:North Richland Hills, MA:Springfield, MD:Gaithersburg, ME:Portlan
```

Q4: cities with >500,000 population for each state (top 3 entries with the largest size of set):

```
TX: [City{name='El Paso', state='TX', population=672538}, City{name='Austin', state='TX', population=
```

```
CA: [City{name='Fresno', state='CA', population=505882}, City{name='San Francisco', state='CA', popul
```

```
AZ: [City{name='Phoenix', state='AZ', population=1488750}, City{name='Tucson', state='AZ', population
```

Q5: population statistics for top 5 states (sorted by state name):

```
AK: avg=298610.00, min=298610, max=298610
```

```
AL: avg=128105.22, min=55996, max=212038
```

```
AR: avg=86332.00, min=58895, max=196537
```

```
AZ: avg=265852.13, min=52819, max=1488750
```

```
CA: avg=150175.77, min=50013, max=3857799
```

Evaluation

The practice will be checked by teachers or SAs. What will be tested:

1. That you understand every line of your own code, not just copy from somewhere
2. That your program compiles correctly (javac)
3. Correctness of the program logic
4. That the result is obtained in a reasonable time

Late submissions after the deadline will incur a 20% penalty, meaning that you can only get 80% of this practice's score.